

Private on-device intelligence

A feasibility and architecture brief for replacing server-side AI with Apple system models and a local MLX fallback - without confusing private inference with private mail.

DECISION	PRIVACY BAR
Proceed with Apple Foundation Models first; use MLX as fallback. Defer MLC.	Do not claim vendor-private mail until Gmail sync and matter state leave the backend.

31 AUGUST 2026 / ENGINEERING + PRODUCT

Scope and assumptions

This report evaluates how to replace off-device AI inference in Electronic Mail with on-device models on Apple platforms, while distinguishing private inference from a genuinely private mail architecture. It assumes the product remains a native Swift application for macOS and iOS and that Gmail is the first provider. "Private" is evaluated relative to the app vendor, AI vendors, network observers, the mail provider, and a person with access to the unlocked device.

Direct answer

Recommended architecture: OS-managed generation first, independent on-device fallback second, deterministic Swift as the authority.

Layer	Recommended choice	Why
Primary generation	Apple SystemLanguageModel	OS-managed, offline, no weight download
Fallback generation	MLX Swift LM	Matches iOS 17/macOS 14; constrained JSON
Embeddings	Small MLX embedder	Versioned semantic retrieval, 17M-33M starting point
Safety authority	Deterministic Swift	Evidence allowlists, validators, provisional merges
MLC	Defer	Use only for cross-platform or model-specific advantage

Yes. Electronic Mail can run its classification, summaries, matter grouping, and review decisions entirely on-device. The preferred stack is:

- 1 Apple SystemLanguageModel as the default on iOS 26/macOS 26 or later when Apple Intelligence is available.
- 2 MLX Swift LM as an optional downloaded-model fallback and as the embedding runtime on the current iOS 17/macOS 14 deployment targets.
- 3 The current deterministic retrieval, validation, evidence allowlisting, provisional state, and shadow-promotion gates retained in Swift.
- 4 MLC LLM only if cross-platform reuse or a specific MLC-supported model justifies its additional build and packaging complexity.

This makes AI inference private, but not the product "truly private" by itself. Today, Gmail content, attachments, tokens, derived matter state, and remote images cross the Electronic Mail backend. To claim that the vendor cannot read mail, Gmail sync and the matter database must move to the device, OAuth must use Google's installed-app flow, and the backend must stop receiving mail content.

What the repository does today

The current architecture is server-owned:

- 1 backend/app/services/ai_inbox.py fetches stored Gmail messages, extracts attachment text, generates an embedding, retrieves candidate matters from Postgres/pgvector, calls a classifier and sometimes a reviewer, validates evidence IDs, and persists the matter assignment.
- 1 Production configuration in backend/app/core/config.py explicitly requires OpenAI for both text and embeddings outside local development.
- 1 MAX_MODEL_TEXT_CHARS is 28,000, with up to 12,000 attachment characters. This exceeds the 4,096-token context documented for the 26.x Apple on-device model once prompts, candidate schemas, and output are included.
- 1 The app already has useful local security foundations: device-only Keychain keys and AES-GCM encryption for cached thread bodies and media in SQLiteLocalMailStore.swift and EncryptedMediaCache.swift.

- 1
- The release privacy manifest conservatively declares email and other user content as collected because the backend processes it.
- The main technical consequence is that a provider swap inside Python is not an on-device implementation. The inference boundary and the durable matter projection need to move into Swift.

Privacy boundaries

1	Private AI inference	No mail content to an AI provider; app backend still sees mail
2	Vendor-private mail	Device talks directly to Google; vendor receives no mail or derived AI state
3	Hardened local mode	Tier 2 plus egress tests, no content telemetry, verified model downloads, remote images blocked

Tier 1: Private AI inference

Mail still syncs through Electronic Mail's backend, but prompts and embeddings no longer go to OpenAI. This removes the AI subprocessor and inference cost, but the vendor's infrastructure can still access mail content. This is a valuable intermediate release, not a "zero knowledge" architecture.

Tier 2: Vendor-private mail

The Apple app talks directly to Google's OAuth and Gmail API endpoints, keeps refresh tokens in the device-only Keychain, persists messages and matter state only in the encrypted local store, and performs inference locally. Electronic Mail's servers receive no email content, attachment content, embeddings, summaries, or matter state. Google still has the mail, and network metadata still exists.

Tier 3: Hardened local mode

Tier 2 plus an egress allowlist, no prompt or content telemetry, crash and diagnostic redaction, remote images disabled by default, model downloads verified by digest, local search indexes encrypted, backup behavior documented, and automated network-capture tests proving that representative mail processing contacts only Google and the user-selected model CDN during installation.

No honest architecture can hide Gmail content from Google, or from malware and users that can access the unlocked app process. "Truly private" should therefore be marketed as "mail content never reaches Electronic Mail or third-party AI servers," not as absolute secrecy.

Runtime options

Runtime	OS floor	Weights	Fit
Apple system model	26+	OS-managed	Best default
MLX Swift LM	iOS 17 / macOS 14	App-managed	Best fallback
MLC LLM	Broad	App-managed + compiled model lib	Cross-platform case
Core AI	27+	App-managed .aimodel	Future option

Apple Foundation Models - recommended default

Apple's Foundation Models framework exposes the OS-managed on-device language model through Swift. Apple states that input and output stay on-device, the model runs offline, and the model does not increase app size. Guided generation maps model output into Swift types, which is a strong fit for the current Pydantic schemas. Availability must be checked at runtime because it depends on supported hardware, region, language, Apple Intelligence being enabled, and model assets being ready.

Strengths:

- 1 Lowest integration and operational burden.
- 1 No model download or model-weight license for the app.
- 1 Native guided generation, sessions, streaming, tools, token counting, and Instruments support.
- 1 Strongest product privacy story and no per-token bill.

Constraints:

- 1 Requires iOS/macOS 26 or later and Apple Intelligence-capable hardware. As of July 2026, supported iPhones start with iPhone 15 Pro; supported Macs require Apple silicon.
- 1 The documented 26.x context is 4,096 tokens. The application must query `contextSize` and never hardcode it because system model versions change with OS updates.
- 1 The OS can rate-limit requests; assets can be unavailable; guardrails can refuse content. Every call needs a deterministic retry/fallback policy.
- 1 Apple periodically updates the model. Prompts and quality gates must be versioned and rerun for each OS/model variant.

MLX Swift LM - recommended fallback and embedding runtime

MLX is the best independent-model path for an Apple-only application. The Swift packages support iOS 17 and macOS 14, exactly matching the current project. `MLXGuidedGeneration` enforces JSON Schema, EBNF, or structural-tag output at token generation time. `MLXEmbedders` supplies small local semantic embedding models. Models can be loaded from a local directory, so production does not need to contact Hugging Face after an authenticated, digest-verified download.

Strengths:

- 1 Works on current deployment targets and does not require Apple Intelligence to be enabled.
- 1 Native Swift, Metal/Apple-silicon focused, with both LLM and embedding packages.
- 1 Constrained JSON output can mirror the existing Pydantic contracts.
- 1 A 17M to 33M parameter embedding model is much lighter than using an LLM for retrieval.

Constraints:

- 1 Electronic Mail owns model selection, model and tokenizer licenses, download integrity, storage, memory admission, thermal behavior, updates, and quality regression testing.
- 1 Model weights add hundreds of megabytes to multiple gigabytes. Download on demand rather than increasing the application bundle.
- 1 Smaller 0.6B to 2B models may not meet the existing 99.5% automatic-merge precision gate. This must be measured rather than assumed.
- 1 The fallback should be optional on low-memory devices. Do not claim universal support just because the OS target builds.

MLC LLM - feasible, not the first choice here

MLC LLM supports Metal on iOS and macOS, OpenAI-style completion APIs, and grammar-constrained JSON. For iOS, model code is compiled ahead of time into a static model library and linked with the TVM runtime; weights may be bundled or downloaded. Its runtime is Apache 2.0, while model weights retain their own licenses.

Use MLC if Electronic Mail plans Android/Web deployment with the same inference engine, or if a required model is substantially better supported by MLC. For this Apple-only Swift codebase, MLX has fewer toolchain layers, supports embeddings in the same package family, and fits the existing deployment targets more naturally.

Core AI and CoreAILanguageModel - future option

Apple's 2026 Core AI stack can export open models into `.aimodel` resources and wrap them in the Foundation Models `LanguageModel` protocol. It offers a compelling unified interface, but currently requires iOS/macOS 27 and Xcode 27. It is a future simplification, not the baseline for the existing iOS 17/macOS 14 product.

Natural Language embeddings - smallest baseline

`NLEmbedding.sentenceEmbedding(for:)` is a built-in local similarity option. It is excellent for a no-download prototype and deterministic fallback, but MLX embeddings provide clearer control over model version, multilingual coverage, vector dimensions, and reproducibility. Benchmark both; use the simplest one that preserves candidate recall.

Recommended on-device pipeline

The current 28,000-character prompt should not be ported verbatim. Use a staged pipeline:

- 1 Parse and sanitize locally. Strip quoted replies, tracking parameters, credentials, authentication codes, and boilerplate before inference. Hash sensitive reference and entity tokens with a device-local account key.
- 2 Extract a compact `MessageEvent` from the newest meaningful message portion. For long bodies or attachments, process bounded chunks in fresh sessions, then deterministically merge the extracted facts.
- 3 Embed the compact event and selected factual text with a small local embedder. Store normalized vectors in the encrypted local matter database.
- 4 Retrieve at most three to five candidates using exact reference, Gmail thread continuity, entity overlap, lexical FTS, recency, and cosine similarity. Retrieval remains deterministic.
- 5 Ask the local model for a constrained `MatterClassification` Swift type. The model can select only supplied candidate IDs and evidence message IDs.
- 6 Run deterministic validators. Reject unknown IDs, unsupported outcome claims, incompatible product/account combinations, and context-free completion states.
- 7 Run a second independent review only for cross-thread merges, close candidates, reopenings, or sensitive outcomes. On a resource-constrained device, mark the result provisional instead of forcing a second generation.
- 8 Persist the decision and evidence locally. Never let model output trigger Gmail mutations; archive, trash, send, and move remain explicit user actions.

This preserves the best part of the existing design: the model proposes semantics, while deterministic code owns authority and safety.

Provider abstraction

Introduce two narrow Swift protocols:

```
protocol LocalTextIntelligence: Sendable {
    var identity: ModelIdentity { get }
    func classify(_ request: ClassificationRequest) async throws
    -> MatterClassification
    func review(_ request: ReviewRequest) async throws
    -> MatterReview
}

protocol LocalEmbeddingProvider: Sendable {
    var identity: ModelIdentity { get }
    var dimensions: Int { get }
    func embed(_ texts: [String]) async throws
    -> [[Float]]
}
```

Implementations:

- 1 `AppleSystemIntelligence`: `SystemLanguageModel` plus `@Generable` schemas on OS 26+.
- 1 `MLXTextIntelligence`: compact pinned model plus MLX guided generation on OS 17+/14+.

- 1 `MLXEmbeddingProvider`: pinned 17M to 33M model, or `NaturalLanguageEmbeddingProvider` for the first prototype.
- 1 `NoAIIntelligence`: deterministic normal inbox behavior when no local model is available.

Persist model ID, weight digest, tokenizer digest, runtime version, prompt version, OS model variant, and output-schema version with every generation. Never mix embeddings from different identities in one index; rebuild in the background when the embedding identity changes.

Moving Gmail off the backend

Google supports installed applications, treats them as unable to keep a client secret, and supports PKCE. The Apple apps can therefore authorize directly and store refresh tokens in the Keychain. A desktop app can use the loopback redirect; iOS should use the Google Sign-In SDK/custom scheme and App Check where appropriate.

For sync, implement Google's documented full sync followed by `history.list` partial sync. Google specifically recommends polling for user-owned devices; Gmail push notifications are designed around a server-side Pub/Sub endpoint. Consequences:

- 1 macOS can maintain a responsive direct sync while running.
- 1 iOS freshness is opportunistic because system background scheduling is not guaranteed. Refresh on app activation and use background tasks for incremental sync and local inference.
- 1 User-initiated historical organization can use a visible long-running task with progress; routine new-message processing should be small and resumable.

The backend can then shrink to the website, optional release/update metadata, and perhaps an OAuth configuration service that never receives authorization codes or tokens. The strongest design needs no account or mail backend at all.

Remote images deserve separate treatment. The current app loads them through an authenticated backend privacy proxy. Removing that proxy either exposes the user's IP to senders or requires images to remain blocked. A vendor-private mode should block remote images by default and make direct loading an explicit choice.

Model distribution, security, and privacy controls

- 1 Keep the application binary small. Download optional MLX weights only after explicit user action.
- 1 Serve an allowlisted manifest over HTTPS containing model ID, exact byte size, SHA-256 digest, runtime compatibility, tokenizer digest, license URL, and minimum memory class.
- 1 Verify the digest before moving weights into the active directory; use atomic installation and preserve the last known-good model until validation succeeds.
- 1 Store weights as read-only application data, exclude replaceable weights from backups, and purge them independently of user mail.
- 1 Encrypt mail, matter state, attachment text, embeddings, and local prompts with the existing account-scoped key pattern. Store refresh tokens only in the device-only Keychain.
- 1 Do not log raw prompts, model transcripts, extracted attachment text, sender addresses, or model output. Do not call model-feedback APIs from production builds.
- 1 Disable analytics in private mode or restrict metrics to coarse on-device counters that are never uploaded without separate consent.
- 1 Add an egress test that processes a seeded mailbox while a network recorder asserts that no request body containing seeded canary strings leaves the process.
- 1 Publish the exact privacy claim and threat model. On-device AI changes App Store privacy disclosures only after the backend data flow is actually removed.

Prototype and rollout plan

Phase 0 - two-week feasibility spike

Build a macOS-only local harness inside the Swift project. It reads a local, encrypted fixture export and runs the existing AI Inbox schemas through:

- 1 SystemLanguageModel on macOS 26+.
- 2 A compact MLX model with constrained JSON.
- 3 NLEmbedding and one 17M to 33M MLX embedder.

Do not modify production sync or the backend. Port only sanitization, candidate retrieval, schemas, validation, and the evaluation harness.

Phase 1 - private inference preview

Run local inference on cached mail while the backend remains the source of messages. Keep results as a device-only shadow projection. Show an explicit "processed on this device" status and retain the normal inbox as fallback.

Phase 2 - local matter authority

Move matter generations, message semantics, membership, decisions, usage/error state, and vectors into SQLite. Stop sending AI prompts and embeddings to the backend. Sync derived matter views between a person's devices only if a separate end-to-end encrypted design is completed; otherwise each device maintains its own projection.

Phase 3 - direct Gmail private mode

Add installed-app OAuth, Keychain token storage, full/partial Gmail sync, direct writes, attachment fetching, remote-image policy, account deletion, and recovery. Remove Gmail content, tokens, mail indexes, and AI projection tables from production infrastructure after a migration and deletion window.

Acceptance gates

Reuse the repository's existing conservative gates and add device-specific gates:

- 1 Explicit-reference candidate recall: 100%.
- 1 Automatic-merge precision: at least 99.5%.
- 1 False automatic merges: zero.
- 1 Unsupported completed/failed headlines: zero.
- 1 Confirmed membership moved automatically: zero.
- 1 Unsolicited Gmail mutation: zero.
- 1 Normal inbox regression: zero.
- 1 Classifier p95: at most 8 seconds on the minimum supported Mac and target iPhone.
- 1 Peak resident memory and thermal state remain within a device-specific budget.
- 1 Cancellation, app backgrounding, low-power mode, model unavailability, guardrail refusal, and corrupted-weight recovery all preserve mail and resume safely.
- 1 Network-capture privacy test: zero canary-bearing egress outside direct Google requests required for mail operations; zero AI content egress.
- 1 Deleting an account removes its Keychain keys, ciphertext, embeddings, projections, downloads scoped to the account, and queued work.

Decision

Proceed, but sequence the work around the privacy claim:

- 1 First prototype SystemLanguageModel plus MLX embeddings on macOS.
- 1 Keep MLX constrained generation as the fallback; do not start with MLC.
- 1 Redesign the prompt into compact staged extraction before measuring model quality.
- 1 Ship "on-device AI" only after network tests prove prompts and embeddings never leave the app.
- 1 Ship "private mail" only after direct Gmail sync eliminates Electronic Mail's server-side mail storage and processing.

The highest risk is not getting a model to run. It is preserving the current high-precision grouping semantics within a small context and under mobile memory/background constraints. The existing shadow generations, review gates, evidence allowlists, and encrypted local cache give the project an unusually strong starting point.

Material limitations and open questions

- 1 No on-device model has yet been benchmarked against this mailbox's labeled examples; runtime recommendations are architectural, not a quality claim.
- 1 App Review treatment of downloaded weights should be confirmed with the chosen packaging before release. Weights must remain data; executable model/runtime code should ship with the signed app.
- 1 Multi-device consistency is intentionally unresolved. Device-local projections maximize privacy but may group the same mail differently after model or OS updates.
- 1 Attachment OCR, non-English mail, encrypted attachments, and very long threads need separate quality datasets.
- 1 Apple SystemLanguageModel changes across OS releases; the app must query capabilities and preserve a no-AI fallback.

Claim-to-source ledger

- 1 Apple, "Meet the Foundation Models framework," WWDC25, accessed 31 August 2026. Supports on-device, offline, private input/output, no app-size increase, guided generation, tools, and sessions. [Open source](#)
- 2 Apple Developer Documentation, "SystemLanguageModel," accessed 31 August 2026. Supports availability states, model variants, runtime checks, and periodic OS updates. [Open source](#)
- 3 Apple Developer Documentation, "Managing the context window," accessed 31 August 2026. Documents the 4,096-token system-model context and chunking guidance. [Open source](#)
- 4 Apple Support, "How to get Apple Intelligence," published 7 July 2026, accessed 31 August 2026. Supports device, storage, language, and regional requirements. [Open source](#)
- 5 Apple, `mlx-swift-lm`, accessed 31 August 2026. Supports iOS 17/macOS 14 packages, MLX LLMs, embedding models, and on-device model loading. [Open source](#)
- 6 Apple, `MLXGuidedGeneration`, accessed 31 August 2026. Supports JSON Schema/EBNF constrained generation on macOS 14/iOS 17+. [Open source](#)
- 7 Apple, `MLXEmbedders` reference, accessed 31 August 2026. Supports local embedders, registered model sizes, batching, pooling, and similarity use. [Open source](#)
- 8 MLC AI, "iOS Swift SDK," accessed 31 August 2026. Supports ahead-of-time model libraries, runtime/static linkage, optional weight bundling, and the Swift API. [Open source](#)
- 9 Apple Developer Documentation, "Running a Core AI model in a Foundation Models session," accessed 31 August 2026. Supports `CoreAILanguageModel` and its iOS/macOS 27 requirement. [Open source](#)
- 10 Apple Developer Documentation, "NLEmbedding," accessed 31 August 2026. Supports local sentence vectors and similarity. [Open source](#)
- 11 Google, "OAuth 2.0 for iOS & Desktop Apps," updated August 2026, accessed 31 August 2026. Supports installed-app OAuth, PKCE, direct token exchange, loopback desktop redirects, and iOS guidance. [Open source](#)

- 12 Google, "Synchronize clients with Gmail," updated August 2026, accessed 31 August 2026. Supports full and partial sync with `history.list`. [Open source](#)
- 13 Google, "Configure push notifications in Gmail API," updated August 2026, accessed 31 August 2026. States polling is recommended for user-owned devices. [Open source](#)
- 14 Apple Developer Documentation, "BGProcessingTask," accessed 31 August 2026. Supports resumable background work while noting that the system may interrupt it. [Open source](#)

Research stopped after the system-model, independent-model, retrieval, Gmail, background, and privacy-boundary claims had primary support, the main alternatives were reconciled, and further searches were unlikely to change the architecture recommendation. No implementation benchmark was performed.